

Informática Avanzada

Tipos de Datos: Primitivos y Objetos

Tipos de datos primitivos

- Almacenan valores numéricos
 - Cada tipo posee un tamaño en bits/bytes
 - Según la forma de almacenamiento
 - Enteros (binario)
 - Punto flotante (signo+exp+magnitud)

Tipo	Tamaño	Rango
<i>boolean</i>	1 bit	true, false
<i>byte</i>	8 bits	-128 a 128
<i>char</i>	16 bits	< ASCII values >
<i>short</i>	16 bits	-32,768 a 32,767
<i>int</i>	32 bits	-2,147,483,648 a 2,147,483,647
<i>long</i>	64 bits	-9,22,372,036,854,775,808 a -9,22,372,036,854,775,807
<i>float</i>	32 bits	Hasta 7 dígitos decimales
<i>double</i>	64 bits	Hasta 16 dígitos decimales

Tipos de datos primitivos

- Almacenan valores numéricos
 - Cada tipo posee un tamaño en bits/bytes
 - Según la forma de almacenamiento
 - Enteros (binario)
 - Punto flotante (signo+exp+magnitud)
- Al declarar una variable de un tipo
 - Reserva espacio en memoria
 - Accede a través del nombre de la variable

```
int diametro;  
diametro = 25;  
float peso = 1.7825f;
```

real $\square$ magnitud $\times 10^{\text{exp}}$

1,626 $\square$ 1.625×10^3
63,412 $\square$ 1.3412×10^4
0.000625 $\square$ 6.25×10^{-4}
0.00735 $\square$ 7.35×10^{-3}

d

Dirección	Identificador	Valor
0x000200	diametro	25 <i>int</i>
0x000201		
0x000202		
0x000203		
0x000204		
0x000205		
0x000206	peso	1.7825 <i>float</i>
0x000207		
0x000208		
0x000209		
0x000210		
0x000211		
0x000212		

Objetos en memoria

- Las Clases establecen la estructura que van a tener sus objetos.
 - Al crear un objeto, este tiene una ubicación en memoria
 - Según su clase, o su “tipo”, los objetos tienen un tamaño determinado

```
new Circulo();
```

```
new MaquinaDeBoletos(25)
```

Dirección	Valor
0xF000A200	
0xF000A201	
0xF000A202	
0xF000A203	
0xF000A204	
0xF000A205	
0xF000A206	
0xF000A207	
0xF000A208	
0xF000A209	
0xF000A210	
0xF000A211	
0xF000A212	

Tipos de Datos de Objetos

- Se usan **variables** para referenciar objetos
- Cada clase define un **tipo** de datos específico
- Las variables de un tipo
 - almacenan **referencias** a objetos de esa clase, no objetos
 - su valor por defecto es **null**
 - no pueden referenciar a objetos de otra clase.

```
Circulo circ;
```

```
circ = new Circulo();
```

```
circ = new MaquinaDeBoletos(25);
```

Dirección	Identificador	Valor
0x000200	circ	
0x000201		
0x000202		
0x000203		
0x000204		
0x000205		
0x000206		

:Circulo



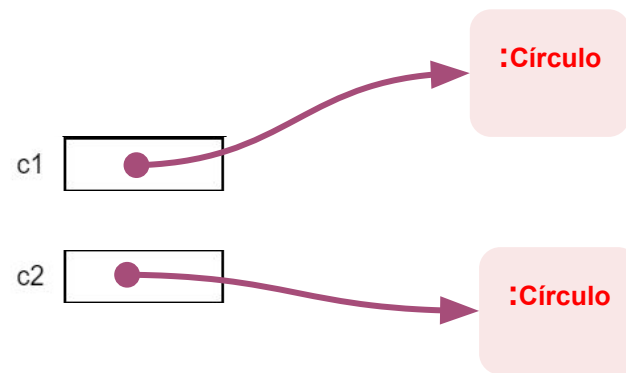
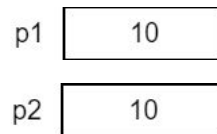
:MaquinaDeBolitos

Tipo Primitivo vs Tipo Objeto - En ejecución

```
int p1 = 10;  
int p2 = 20;  
System.out.println(p1 == p2); // --false
```

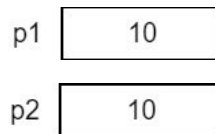
```
p2 = 10;  
System.out.println(p1 == p2); // --true
```

```
Circulo c1;  
Circulo c2;  
c1 = new Circulo();  
c2 = new Circulo();  
System.out.println(c1 == c2); // -> false
```



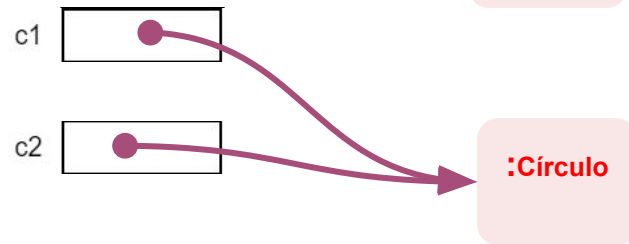
Tipo Primitivo vs Tipo Objeto - En ejecución

```
int p1 = 10;  
int p2 = 20;  
System.out.println(p1 == p2); // --false
```



```
p2 = 10;  
System.out.println(p1 == p2); // --true
```

```
Circulo c1;  
Circulo c2;  
c1 = new Circulo();  
c2 = new Circulo();  
System.out.println(c1 == c2); // -> false  
c1 = c2;  
System.out.println(c1 == c2); // -> true
```



Informática Avanzada

Interacción entre Objetos

Abstracción y Modularización

- Los objetos se pueden componer de otros objetos
- El diseño de los objetos debe considerar
 - Separar en partes, definir la función y forma de interactuar de cada parte, y construir en forma independiente (**modularización**)
 - Ignorar los detalles internos de los objetos y sólo entender cómo usarlos (**abstracción**)



Proyecto Reloj Digital

- Modelo de un reloj digital
- Muestra la hora en un display
- Podemos crear una única clase
- O modelar un visor compuesto displays mas pequeño

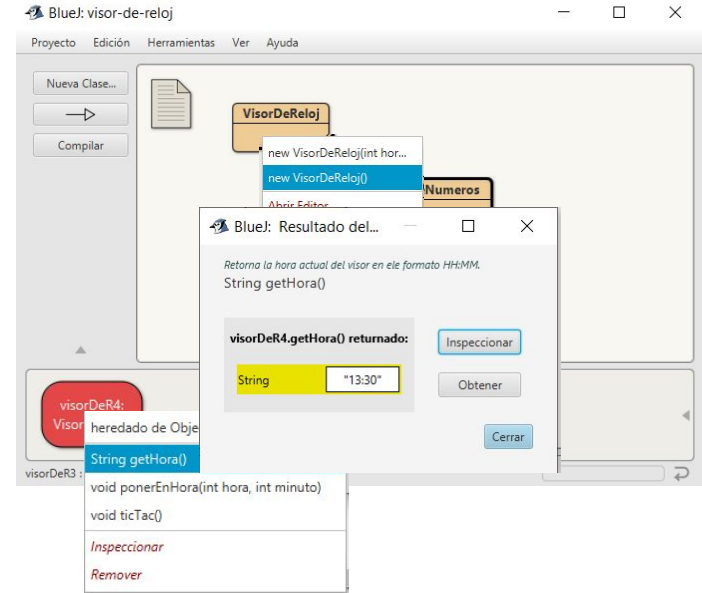
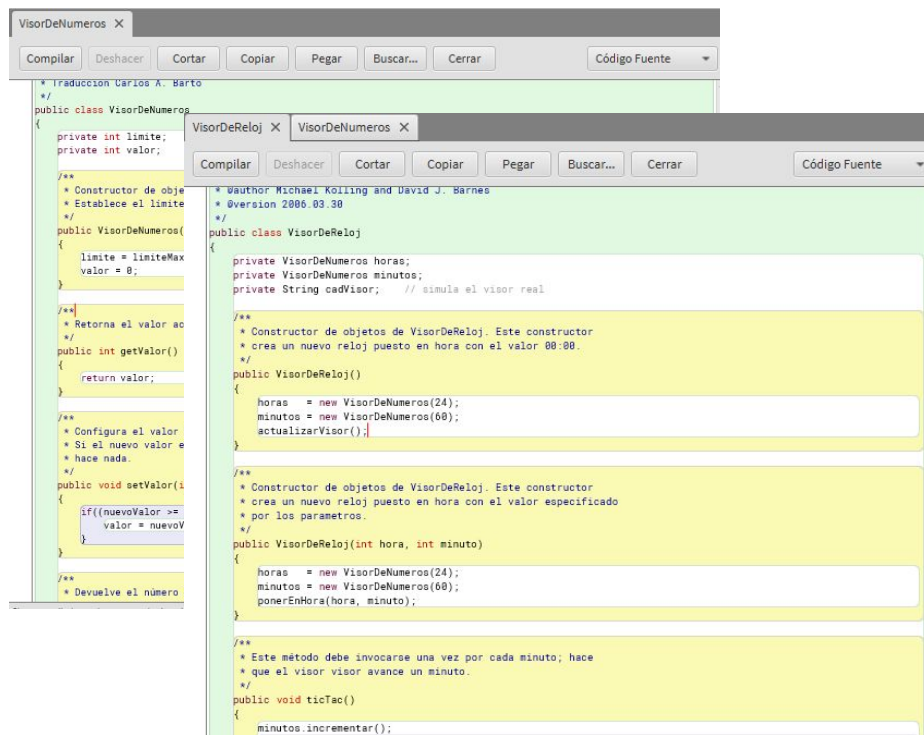


Diagrama De Clases



The screenshot shows two IDE windows. The top window displays the `VisorDeNumeros` class with the following code:

```
/**
 * Traducción Carlos A. Barte
 */
public class VisorDeNumeros {
    private int limite;
    private int valor;

    /**
     * Constructor de objeto
     * Establece el limite
     */
    public VisorDeNumeros() {
        limite = limiteMax;
        valor = 0;
    }

    /**
     * Retorna el valor actual
     */
    public int getValor() {
        return valor;
    }

    /**
     * Configura el valor
     * Si el nuevo valor es mayor
     * que el actual, se actualiza.
     */
    public void setValor(int nuevoValor) {
        if (nuevoValor >= valor) {
            valor = nuevoValor;
        }
    }

    /**
     * Devuelve el número
     */
}
```

The bottom window displays the `VisorDeReloj` class with the following code:

```
/**
 * Author Michael Kolling and David J. Barnes
 * Version 2006.03.30
 */
public class VisorDeReloj {
    private VisorDeNumeros horas;
    private VisorDeNumeros minutos;
    private String cadVisor; // simula el visor real

    /**
     * Constructor de objetos de VisorDeReloj. Este constructor
     * crea un nuevo reloj puesto en hora con el valor 00:00.
     */
    public VisorDeReloj() {
        horas = new VisorDeNumeros(24);
        minutos = new VisorDeNumeros(60);
        actualizarVisor();
    }

    /**
     * Constructor de objetos de VisorDeReloj. Este constructor
     * crea un nuevo reloj puesto en hora con el valor especificado
     * por los parametros.
     */
    public VisorDeReloj(int hora, int minuto) {
        horas = new VisorDeNumeros(24);
        minutos = new VisorDeNumeros(60);
        ponerEnHora(hora, minuto);
    }

    /**
     * Este método debe invocarse una vez por cada minuto; hace
     * que el visor visor avance un minuto.
     */
    public void ticTac() {
        minutos.incrementar();
    }
}
```

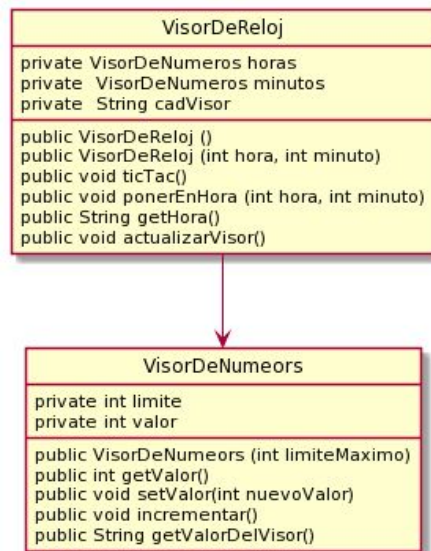


Diagrama de Objetos

```
public class VisorDeReloj
{
    private VisorDeNumeros horas;
    private VisorDeNumeros minutos;
    private String cadVisor;    // simula el visor real

    /**
     * Constructor de objetos de VisorDeReloj. Este constructor
     * crea un nuevo reloj puesto en hora con el valor 00:00.
     */
    public VisorDeReloj()
    {
        horas = new VisorDeNumeros(24);
        minutos = new VisorDeNumeros(60);
        actualizarVisor();
    }
}
```

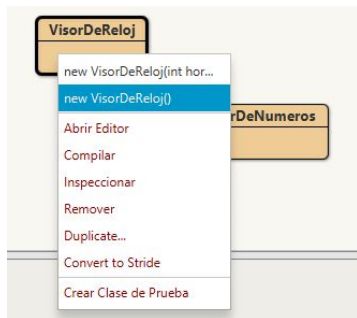
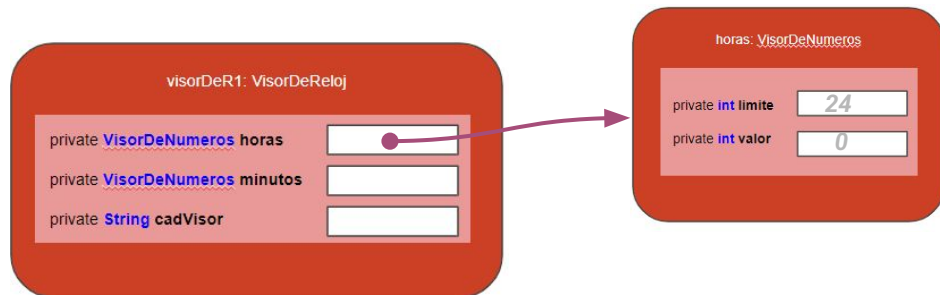


Diagrama de Objetos

```
public class VisorDeReloj
{
    private VisorDeNumeros horas;
    private VisorDeNumeros minutos;
    private String cadVisor;    // simula el visor real

    /**
     * Constructor de objetos de VisorDeReloj. Este constructor
     * crea un nuevo reloj puesto en hora con el valor 00:00.
     */
    public VisorDeReloj()
    {
        horas = new VisorDeNumeros(24);
        minutos = new VisorDeNumeros(60);
        actualizarVisor();
    }
}
```

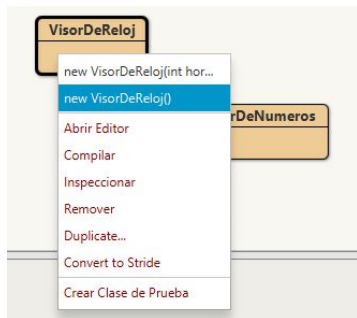
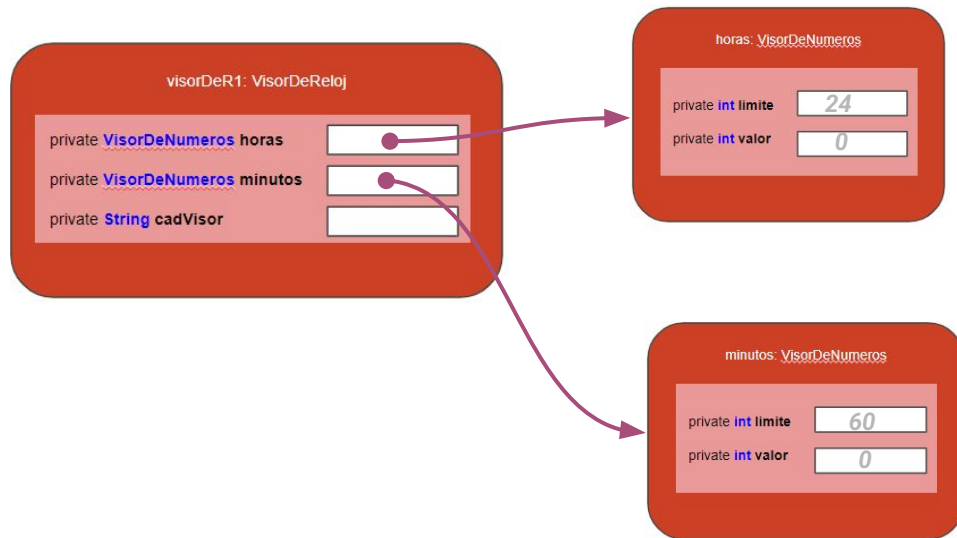
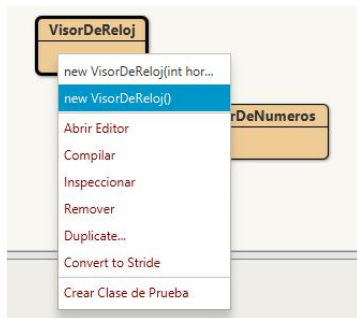
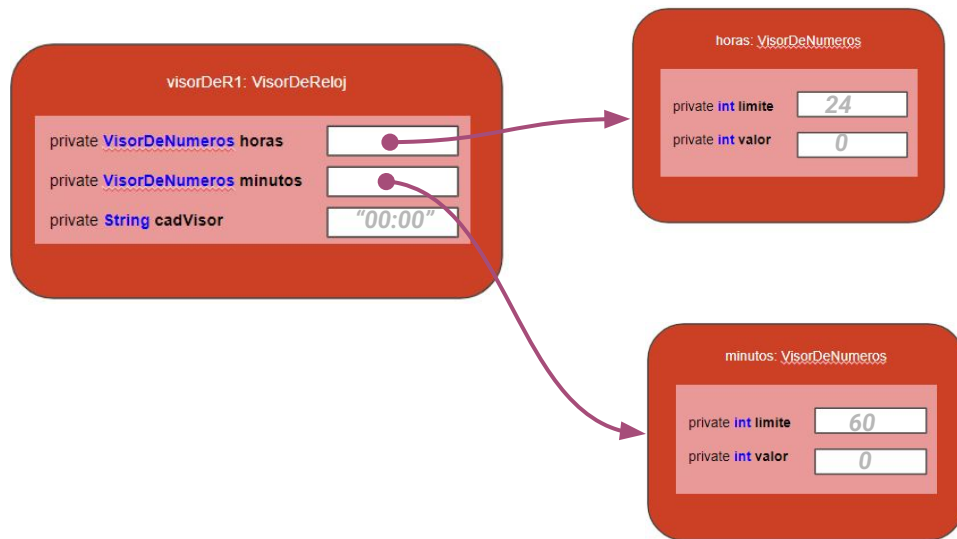


Diagrama de Objetos

```
public class VisorDeReloj
{
    private VisorDeNumeros horas;
    private VisorDeNumeros minutos;
    private String cadVisor;    // simula el visor real

    /**
     * Constructor de objetos de VisorDeReloj. Este constructor
     * crea un nuevo reloj puesto en hora con el valor 00:00.
     */
    public VisorDeReloj()
    {
        horas = new VisorDeNumeros(24);
        minutos = new VisorDeNumeros(60);
        actualizarVisor();
    }
}
```



Llamar a métodos de otros objetos

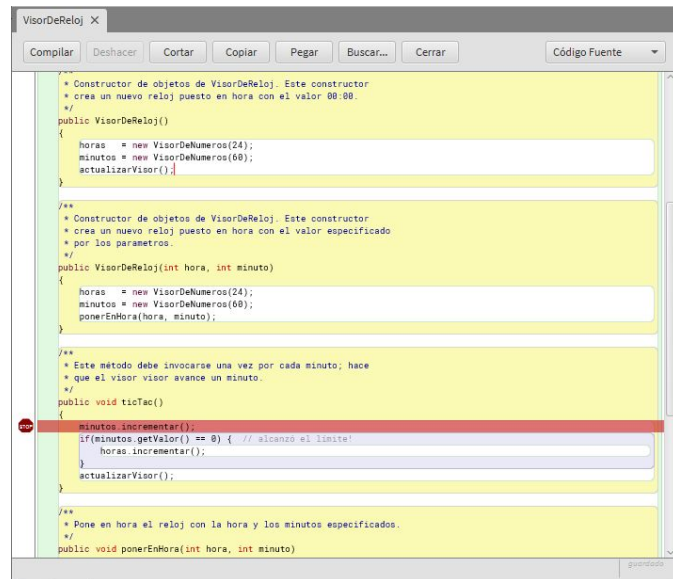
- Los métodos se invocan sobre objetos concretos
- Para acceder a un objeto, lo hacemos a través de una variable que lo referencie
- Utilizamos el operador “.” para acceder a sus métodos y variables públicas
 - hora.limite
 - hora.setValor();
- Para referenciarse a si mismo, en vez de variable usamos “this”
 - this.limite

```
*/  
public void setValor(int nuevoValor)  
{  
    if((nuevoValor >= 0) && (nuevoValor < limite)) {  
        valor = nuevoValor;  
    }  
}
```

```
/**  
 * Este método debe invocarse una vez por cada minuto; hace  
 * que el visor visor avance un minuto.  
 */  
public void ticTac()  
{  
    minutos.incrementar();  
    if(minutos.getValor() == 0) { // alcanzó el límite!  
        horas.incrementar();  
    }  
    actualizarVisor();  
}  
  
/**  
 * Pone en hora el reloj con la hora y los minutos especificados.  
 */  
public void ponerEnHora(int hora, int minuto)  
{  
    horas.setValor(hora);  
    minutos.setValor(minuto);  
    actualizarVisor();  
}
```

Debugger

- Herramienta de desarrollo
- Permite
 - Controlar la ejecución del programa
 - Visualizar el estado del sistema
 - Variables de instancia,
 - Variables locales
 - Variables de clase
 - Secuencia de ejecución
 - Establecer “breakpoints”



The screenshot shows a Java IDE window titled 'VisorDeReloj X'. The menu bar includes 'Compilar', 'Deshacer', 'Cortar', 'Copiar', 'Pegar', 'Buscar...', and 'Cerrar'. A dropdown menu for 'Código Fuente' is visible. The code is for a clock application. A red breakpoint icon is set on the line `minutos.incrementar();` within the `ticTac()` method. The code includes two constructors and two methods: `ticTac()` and `ponerEnHora()`.

```
/* Constructor de objetos de VisorDeReloj. Este constructor
 * crea un nuevo reloj puesto en hora con el valor 00:00.
 */
public VisorDeReloj()
{
    horas = new VisorDeNumeros(24);
    minutos = new VisorDeNumeros(60);
    actualizarVisor();
}

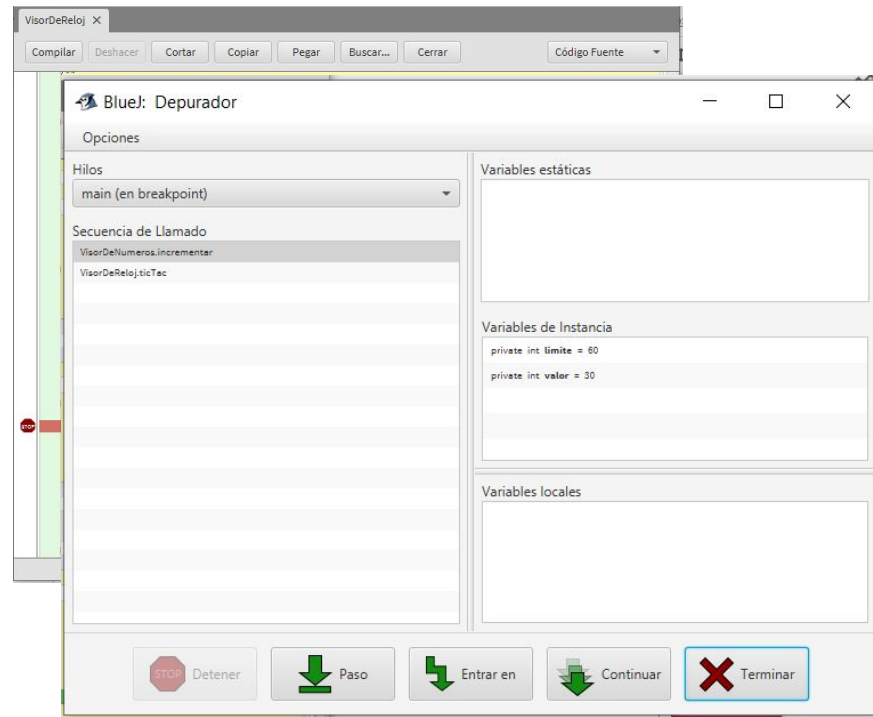
/**
 * Constructor de objetos de VisorDeReloj. Este constructor
 * crea un nuevo reloj puesto en hora con el valor especificado
 * por los parametros.
 */
public VisorDeReloj(int hora, int minuto)
{
    horas = new VisorDeNumeros(24);
    minutos = new VisorDeNumeros(60);
    ponerEnHora(hora, minuto);
}

/**
 * Este método debe invocarse una vez por cada minuto; hace
 * que el visor visor avance un minuto.
 */
public void ticTac()
{
    minutos.incrementar();
    if(minutos.getValor() == 0) { // alcanzo el limite!
        horas.incrementar();
    }
    actualizarVisor();
}

/**
 * Pone en hora el reloj con la hora y los minutos especificados.
 */
public void ponerEnHora(int hora, int minuto)
```

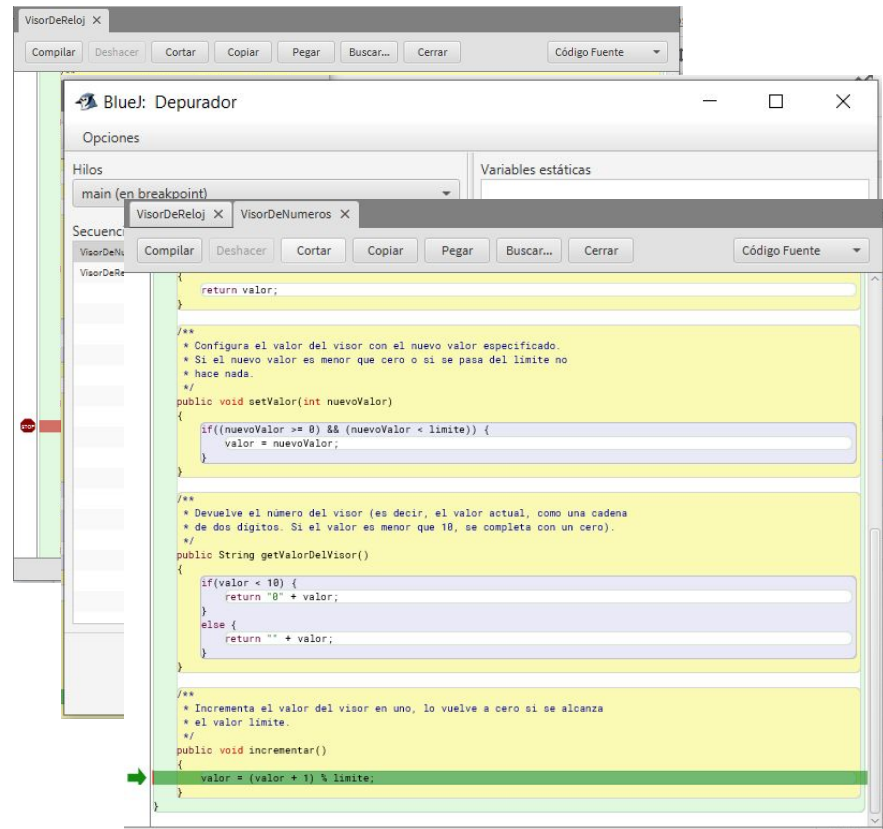

Debugger

- Herramienta de desarrollo
- Permite
 - Controlar la ejecución del programa
 - Visualizar el estado del sistema
 - Variables de instancia,
 - Variables locales
 - Variables de clase
 - Secuencia de ejecución
 - Establecer "breakpoints"

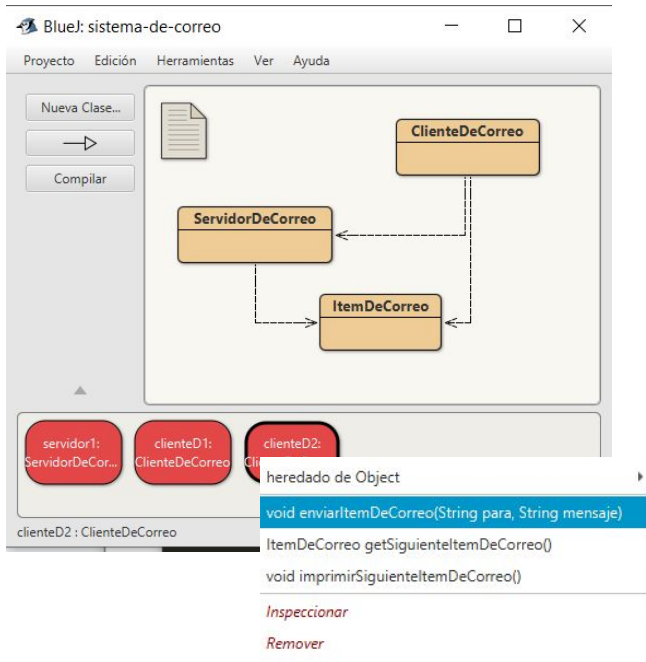


Debugger

- Herramienta de desarrollo
- Permite
 - Controlar la ejecución del programa
 - Visualizar el estado del sistema
 - Variables de instancia,
 - Variables locales
 - Variables de clase
 - Secuencia de ejecución
 - Establecer "breakpoints"



Proyecto Sistema De Correos



```
ItemDeCorreo x
Compilar Deshacer Cortar Copiar Pegar Buscar... Cerrar Código Fuente
* @version 2000.03.30
*/
public class ItemDeCorreo
{
    // El remitente del item.
    private String de;
    // El destinatario.
    private String para;
    // The text of the message.
    private String mensaje;

    /**
     * Crea un item de mail desde el servidor para el destinatario
     * dado, conteniendo el mensaje dado.
     * @param de El remitente de este item.
     * @param para El destinatario de este item.
     * @param mensaje El texto del mensaje a ser enviado.
     */
    public ItemDeCorreo(String de, String para, String mensaje)
    {
        this.de = de;
        this.para = para;
        this.mensaje = mensaje;
    }
}
```



<http://www.plantuml.com/>

[illegible]

